Introduction to Programming in C++
Seventh Edition

Chapter 8: More on the Repetition Structure

Objectives

- Include a posttest loop in pseudocode
- Include a posttest loop in a flowchart
- Code a posttest loop using the C++ `do while` statement
- Nest repetition structures

Posttest Loops

- Repetition structures can be either pretest or posttest loops
- Pretest loop – condition evaluated before instructions are processed
- Posttest loop – condition evaluated after instructions are processed
- Posttest loop's instructions are always processed at least once
- Pretest loop's instructions may never be processed

Posttest Loops (cont'd.)

Problem specification
Sherri is standing an unknown number of steps away from the Burlington fountain. Write the instructions that direct Sherri to walk from her current location to the fountain.

Illustration A

Illustration B

Algorithm 1 – pretest loop

```
repeat while (you are not directly in front of the fountain)
      walk forward one complete step
end repeat
```

condition

works when Sherri is zero or more steps away from the fountain

Algorithm 2 – posttest loop

```
repeat
      walk forward one complete step
end repeat while (you are not directly in front of the fountain)
```

works only when Sherri is at least one step away from the fountain

condition

Figure 8-1 Problem specification, illustrations, and solutions containing pretest and posttest loops

Posttest Loops (cont'd.)

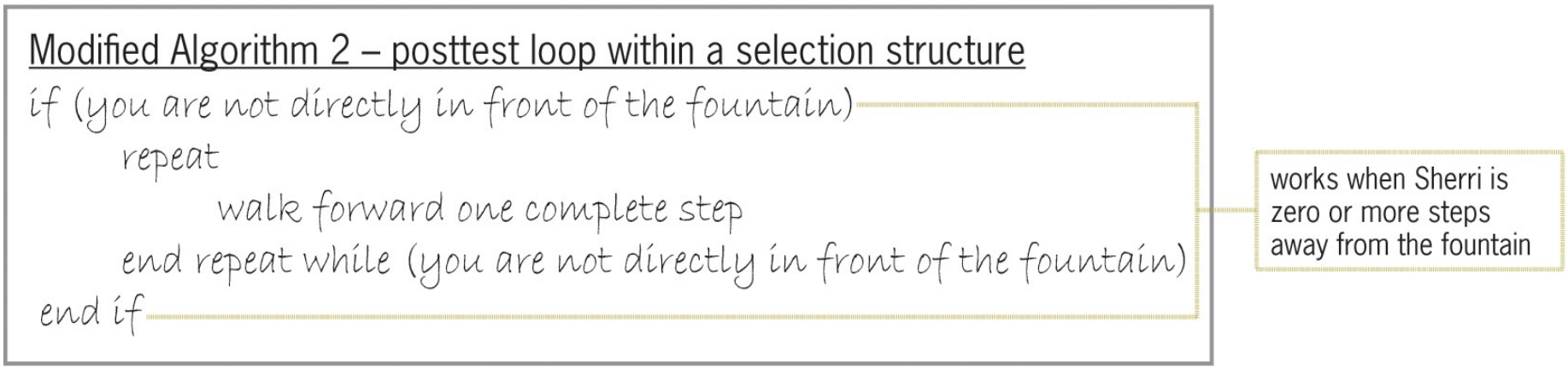

Figure 8-2 Selection structure added to Solution 2 from Figure 8-1

Flowcharting a Posttest Loop

- Decision symbol in a flowchart (a diamond) representing a repetition structure contains the loop condition
- Decision symbol appears at the top of a pretest loop, but at the bottom of a posttest loop

Flowcharting a Posttest Loop (cont'd.)

Problem specification
Create a program that calculates the commission for each of a company's salespeople. The commission is calculated by multiplying the salesperson's sales amount by 20%.

Algorithm 1 (pretest loop):

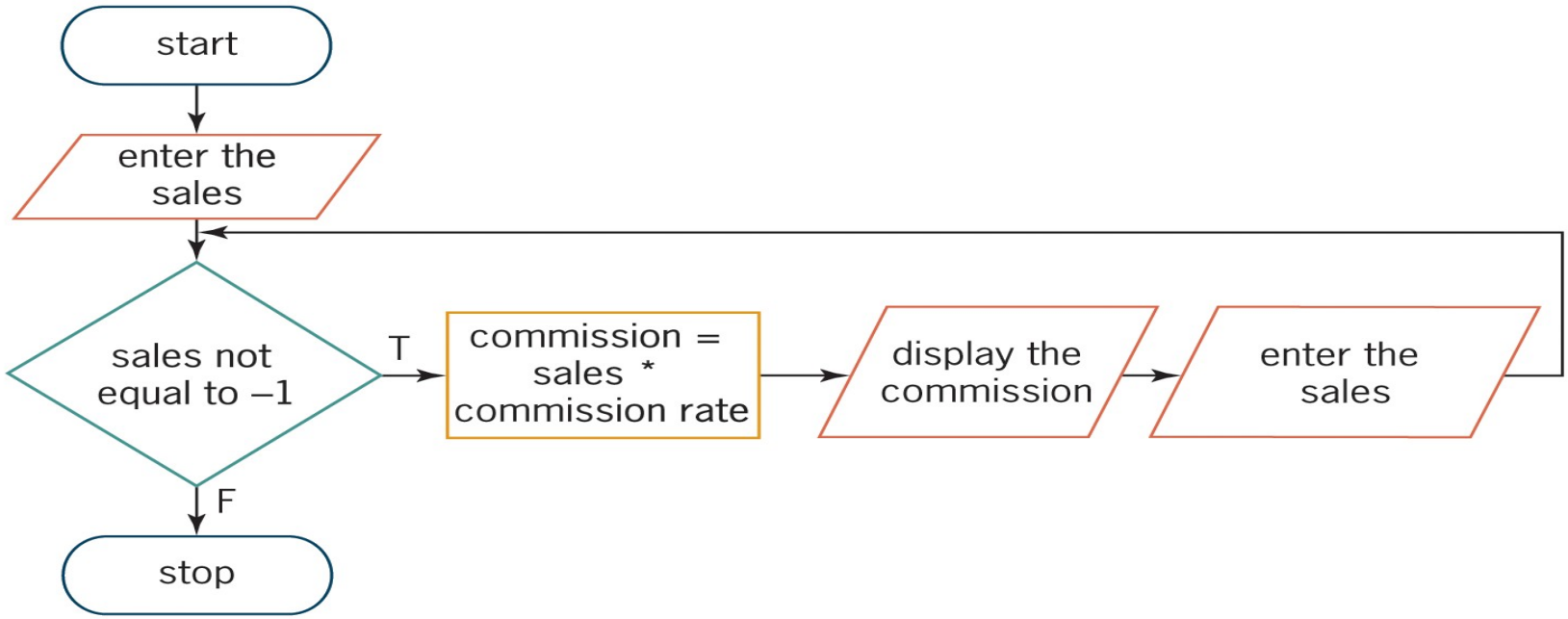

Figure 8-3 Commission Program's problem specification and flowcharts

Flowcharting a Posttest Loop (cont'd.)

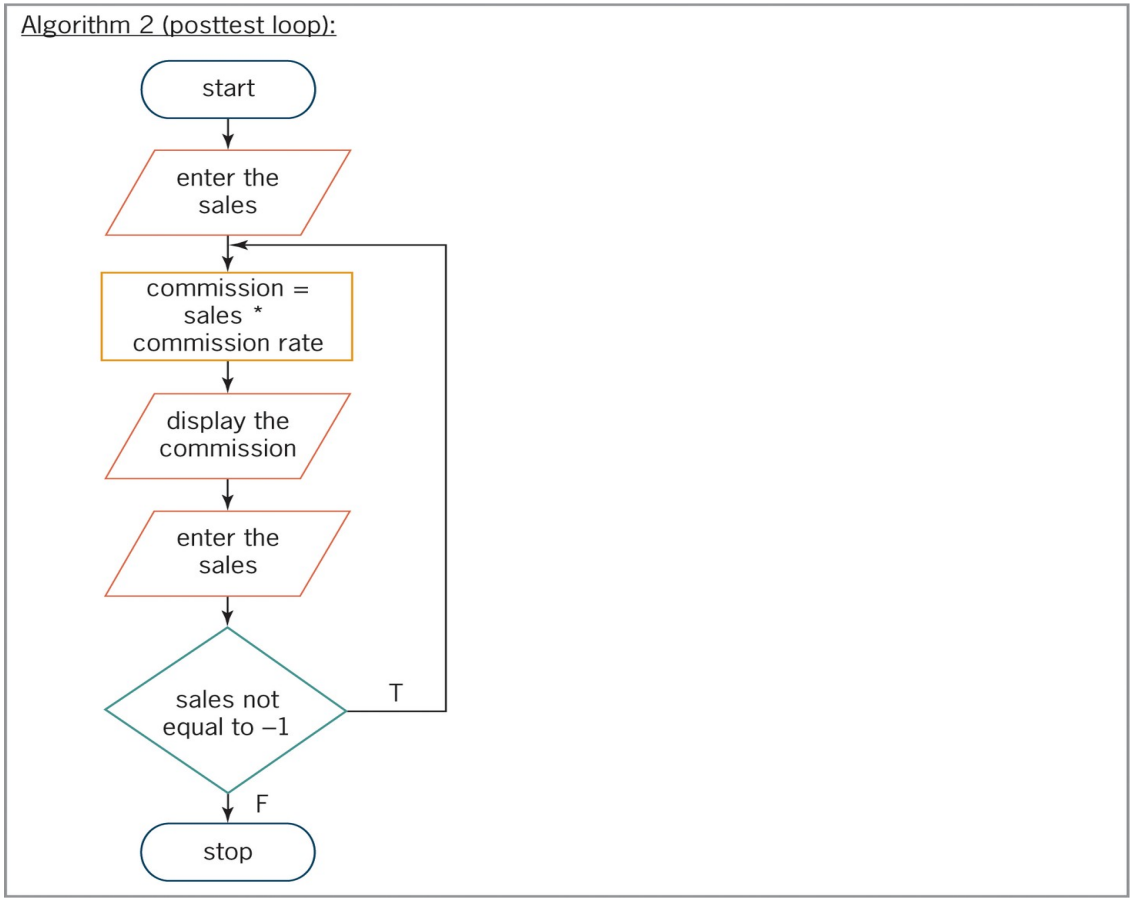

Figure 8-3 Commission Program's problem specification and flowchart

The `do while` Statement

- `do while` statement is used to code posttest loops in C++
- Syntax:

```
do {
        one or more statements to be processed one time,
        and thereafter as long as the condition is true
} while (condition);
```

- Some programmers use a comment (such as: `//begin loop`) to mark beginning of loop

The `do while` Statement (cont'd.)

- Programmer must provide loop *condition*
  - Must evaluate to a Boolean value
  - May contain variables, constants, functions, arithmetic operators, comparison operators, and logical operators
- Programmer must also provide statements to be executed when *condition* evaluates to true
- Braces are required around statements if there are more than one

The `do while` Statement (cont'd.)

HOW TO Use the do while Statement

Syntax

```
do  //begin loop
{
    one or more statements to be processed one time, and
    thereafter as long as the condition is true
}   while (condition);
```

the statement ends with a semicolon

Example 1

```
int hours = 0;

cout << "Hours worked: ";
cin >> hours;
do    //begin loop
{
      cout << "Gross pay: $" << hours * 10 << endl << endl;
      cout << "Hours worked: ";
      cin >> hours;
}   while (hours > 0);
```

priming read

update read

semicolon

Example 2

```
char another = ' ';
double number = 0.0;
double sum = 0.0;

cout << "Enter a number? (Y/N) ";
cin >> another;
do  //begin loop
{
      cout << "Number: ";
      cin >> number;
      sum += num;
      cout << "Enter another number? (Y/N)";
      cin >> another;
}   while (toupper(another) == 'Y');
```

priming read

update read

semicolon

Figure 8-4 How to use the `do while` statement

The `do while` Statement (cont'd.)

IPO chart information

Input

commission rate (20%)
sales

Processing

none

Output

commission

Algorithm

enter t e sales

2 re eat

calc late t e commission m lti l in t e sales t e commission rate

is la t e commission

enter t e sales

en re eat ile (t e sales are not e al to)

C++ instructions

```
const double RATE = 0.2;
int sales = 0;

double commission = 0.0;

cout << "First salesperson's sales
(-1 to stop): ";
cin >> sales;

do    //begin loop
{
   commission = sales * RATE;
   cout << "Commission: $"
   << commission << endl << endl;
   cout << "Next salesperson's
   sales (-1 to stop): ";
   cin >> sales;
} while(sales != -1);
```

```
Commission
First salesperson's sales (-1 to stop): 1200
Commission: $240

Next salesperson's sales (-1 to stop): 800
Commission: $160

Next salesperson's sales (-1 to stop): -1
Press any key to continue . . .
```

Figure 8-5 Commission Program containing a posttest loop

Nested Repetition Structures

- Like selection structures, repetition structures can be nested
- You can place one loop (the inner, or **nested loop**) inside another loop (the outer loop)
- Both loops can be pretest loops or posttest loops, or the two loops may be different types
- Programmer decides whether a problem requires a nested loop by analyzing the problem specification

Nested Repetition Structures (cont'd.)

Problem specification and algorithm

Trixie works as a waitress at a local diner. The diner just opened for the day, and there are customers already sitting at several of the tables. Write the instructions that direct Trixie to go to each table that needs to be waited on and tell the customers about the daily specials.

follow these instructions for each table

```
repeat for (each table that needs to be waited on)
        go to a table that needs to be waited on
        tell the customers at the table about the daily specials
end repeat
```

Figure 8-6 Problem specification and solution that requires a loop

Nested Repetition Structures (cont'd.)

Problem specification and algorithm

Trixie works as a waitress at a local diner. The diner just opened for the day, and there are customers already sitting at several of the tables. Write the instructions that direct Trixie to go to each table that needs to be waited on and tell the customers about the daily specials. While at each table, Trixie should take each customer's order. She should then submit the order for the entire table to the cook.

```
repeat for (each table that needs to be waited on)
        go to a table that needs to be waited on
        tell the customers at the table about the daily specials
        repeat for (each customer at the table)
                ask the customer for his or her order
                record the order on the order slip for that table
        end repeat
        go over to the cook at the counter
        tear the appropriate order slip from the order pad
        give the order slip to the cook
end repeat
```

follow these instructions for each customer at the current table

follow these instructions for each table

Figure 8-7 Modified problem specification and solution that requires a nested loop

The Clock Program

- Simple program that simulates a clock with minutes and seconds.
- The outer loop represents minutes.
- The inner loop (nested loop) represents seconds.
- To make it easier to desk-check the instructions, the nested loop uses only three seconds per minute and the outer loop stops after two minutes.

The Clock Program (cont'd.)

```
1. start minutes at 0
2. repeat while (minutes are less than 60)
        start seconds at 0
        repeat while (seconds are less than 60)
              move second hand 1 position, clockwise
              add 1 to seconds
        end repeat
        move minute hand 1 position, clockwise
        add 1 to minutes
   end repeat
```

outer loop

```
int main()
{
     cout << "Minutes     Seconds" << endl;
     for (int minutes = 0; minutes < 2; minutes += 1)
     {
          for (int seconds = 0; seconds < 3; seconds += 1)
               cout << "    " << minutes
                    << "           " << seconds << endl;
          //end for
          cout << endl;
     } //end for
     return 0;
}    //end of main function
```

outer loop

```
Clock
Minutes     Seconds
    0           0
    0           1
    0           2

    1           0
    1           1
    1           2

Press any key to continue . . .
```

Figure 8-8 Algorithm, code, and a sample run of the Clock Program

The Clock Program (cont'd.)

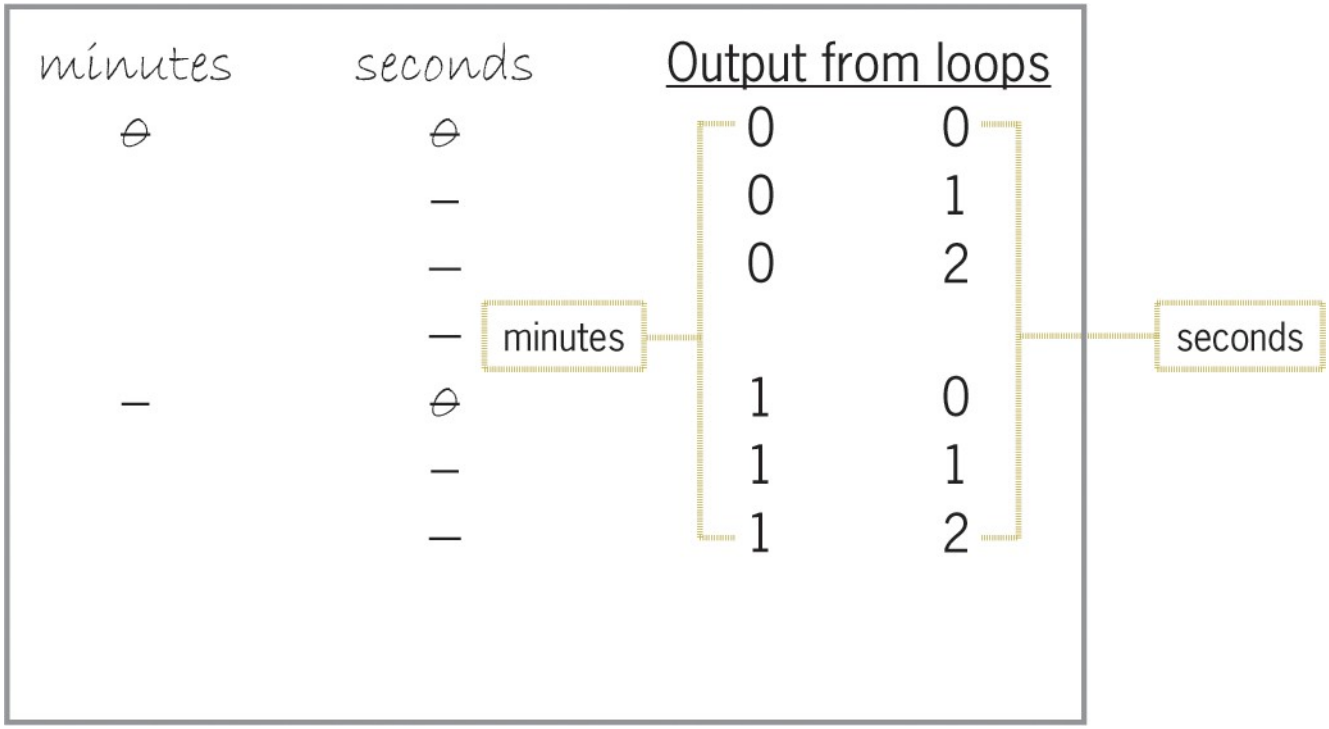

Figure 8-10 Completed desk-check table and output

The Car Depreciation Program

- Calculate the depreciation of a car over time.
- Program uses a counter-controlled loop to display the value of a new car at the end of each of five years, using a 15% annual depreciation rate.

Car Depreciation Program (cont'd.)

Problem specification

Create a program that displays the value of a new car at the end of each of five years, using a 15% annual depreciation rate.

```
int main()
{
    double originalValue = 0.0;
    double depreciation = 0.0;
    double currentValue = 0.0;

    cout << "Original value: ";
    cin >> originalValue;
    cout << endl << fixed << setprecision(0);

    cout << "Depreciation rate: 15%" << endl;
    cout << "Value after year: " << endl;
```

Figure 8-11 Beginning of Car Depreciation Program with Problem Specification

Car Depreciation Program (cont'd.)

```
    currentValue = originalValue;
    for (int year = 1; year < 6; year += 1)
    {
        depreciation = currentValue * 0.15;
        currentValue -= depreciation;
        cout << year << "   $" << currentValue << endl;
    } //end for

    return 0;
}   //end of main function
```

loop

```
Car Depreciation
Original value: 25000

Depreciation rate: 15%
Value after year:
1   $21250
2   $18063
3   $15353
4   $13050
5   $11093
Press any key to continue . . .
```

Figure 8-11 Remainder of Car Depreciation Program with sample run

Car Depreciation Program (cont'd.)

- Modify the Car Depreciation Program to use different depreciation rates.
- This modified program displays the value of a new car at the end of each of five years, using annual depreciation rates of 15%, 20%, and 25%.

Car Depreciation Program (cont'd.)

Problem specification
Create a program that displays the value of a new car at the end of each of five years, using annual depreciation rates of 15%, 20%, and 25%.

```
int main()
{
    double originalValue = 0.0;
    double depreciation = 0.0;
    double currentValue = 0.0;

    cout << 'Original value: ';
    cin >> originalValue;
    cout << endl << fixed << setprecision(0);
```

Figure 8-12 Beginning of the modified Car Depreciation Program with problem specification

Car Depreciation Program (cont'd.)

outer loop

```
    for (double rate = 0.15; rate < 0.26; rate += 0.05)
    {
        cout << "Depreciation rate: " << rate * 100 << "%" << endl;
        cout << "Value after year: " << endl;

        currentValue = originalValue;
        for (int year = 1; year < 6; year += 1)
        {
            depreciation = currentValue * rate;
            currentValue -= depreciation;
            cout << year << "   $" << currentValue << endl;
        }  //end for
        cout << endl;
    }  //end for

    return 0;
}   //end of main function
```

```
Car Depreciation
Original value: 25000

Depreciation rate: 15%
Value after year:
1   $21250
2   $18063
3   $15353
4   $13050
5   $11093

Depreciation rate: 20%
Value after year:
1   $20000
2   $16000
3   $12800
4   $10240
5   $8192

Depreciation rate: 25%
Value after year:
1   $18750
2   $14063
3   $10547
4   $7910
5   $5933

Press any key to continue . . .
```

Figure 8-12 Remainder of the modified Car Depreciation Program with sample run

Car Depreciation Program (cont'd.)

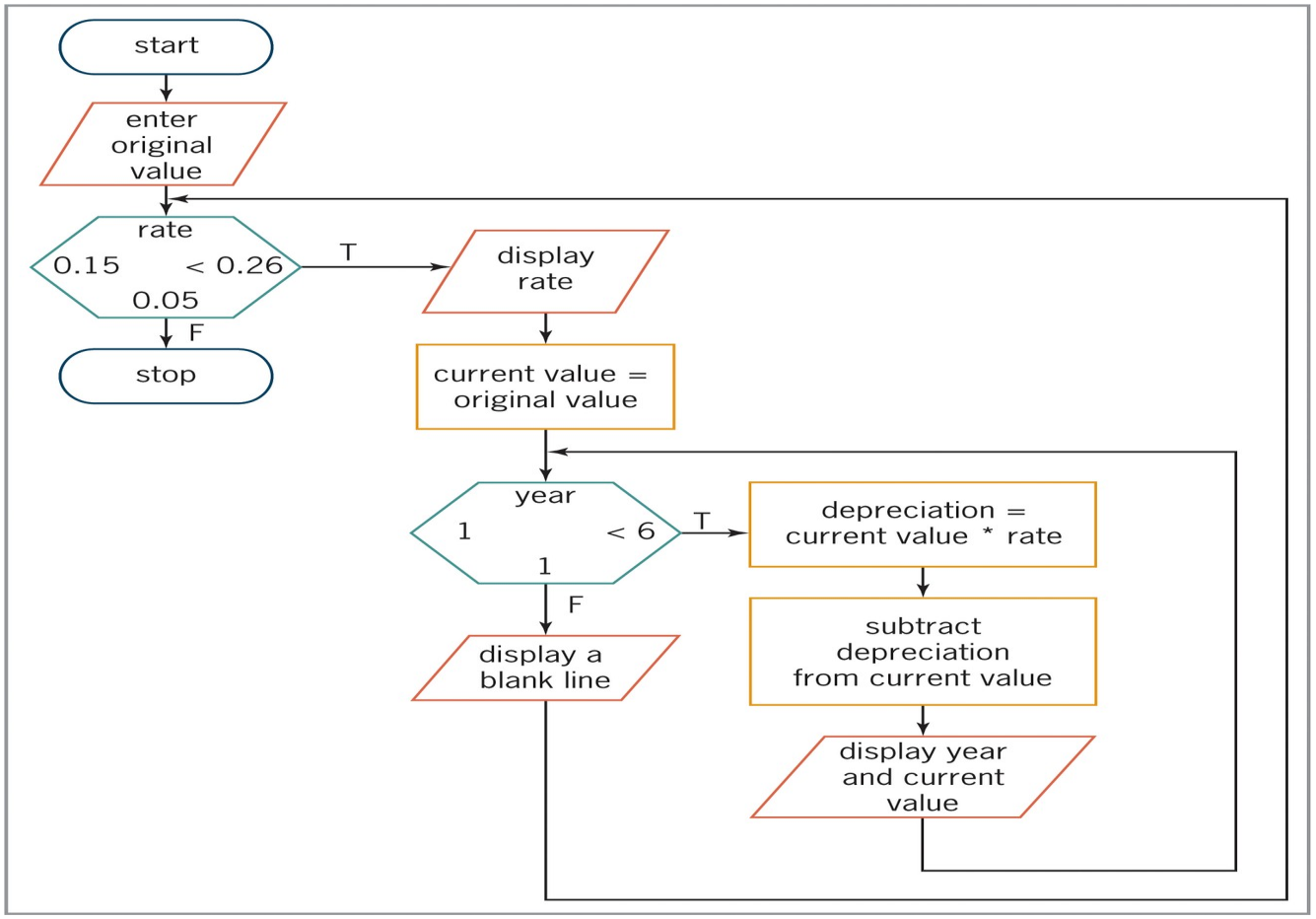

Figure 8-13 Flowchart for the modified Car Depreciation Program

Summary

- A repetition structure can be either a pretest loop or a posttest loop
- In a pretest loop, the loop condition is evaluated *before* the instructions in the loop are processed
- In a posttest loop, the evaluation occurs *after* the instructions within the loop are processed
- Use the `do while` statement to code a posttest loop in C++
- Use either the `while` statement or the `for` statement to code a pretest loop in C++

Summary (cont'd.)

- Repetition structures can be nested, which means one loop (called the inner or nested loop) can be placed inside another loop (called the outer loop)
- For nested repetition structures to work correctly, the entire inner loop must be contained within the outer loop

Lab 8-1: Stop and Analyze

- Study the program below and answer the questions.

```
 1 //Lab8-1.cpp
 2 //Created/revised by <your name> on <current date>
 3
 4 #include <iostream>
 5 using namespace std;
 6
 7 int main()
 8 {
 9     int maxRows = 0;
10
11     cout << "Maximum number of rows: ";
12     cin >> maxRows;
13
14     for (int row = 0; row < maxRows; row += 1)
15     {
16         for (int space = 0; space < maxRows - row; space += 1)
17               cout << " ";
18         //end for
19
20         for (int asterisk = 0; asterisk <= row; asterisk += 1)
21               cout << "* ";
22         //end for
23
24         cout << endl;
25     }   //end for
26     return 0;
27 }   //end of main function
```

notice the space

Figure 8-14 Code for Lab 8-1

Lab 8-2: Plan and Create

Problem specification

Create a program that allows the user to enter a person's age (in years) and current salary. Both input items should be entered as integers. The program should display a person's total earnings before retirement at age 65, using annual raise rates of 3%, 4%, and 5%. Display the total earning amounts as integers.

Input	Processing	Output
age (1-64 years) current salary	Processing items: raise rate (3%, 4%, 5%) years until retirement new salary year (counter)	total earnings (at end of each of the years until retirement)

```
Algorithm:
1. enter the age
2. if (the age is less than 1 or greater than 64)
       display reenter message
   else
       enter current salary
       calculate years until retirement = 65 - age
       repeat for (each raise rate)
           assign current salary to new salary
           assign current salary to total earnings
           repeat for (year 2 to years until retirement)
                 new salary = new salary * (1 + raise rate)
                 add new salary to total earnings
           end repeat
           display raise rate and total earnings
       end repeat
   end if
```

at this point, the current salary is year 1's salary

current age	current salary	raise rate	years until retirement
62	25000	~~.03~~ ~~.04~~ ~~.05~~ .06	3

Figure 8-15 Problem specification, IPO chart, and desk-check table for the Retirement program

Lab 8-2: Plan and Create (cont'd.)

Figure 8-15 Remainder of the desk-check table for the Retirement program

Lab 8-2: Plan and Create (cont'd.)

IPO chart information

Input

age (1-64 years)
current salary

Processing

ra se rate (4)
years unt l ret re ent
ne salary
year (c unter)

Output

t tal earn ngs (at en
eac t e years unt l
ret re ent)

Algorithm

1 enter t e age

(t e age s less t an 1 r greater)t an 64
s lay reenter e sag

else
enter current salary

calculate years unt l ret re ent 6
re eat r (eac ra se rate)

C++ instructions

```
int age = 0;
int currentSalary = 0;

t    s   ar  a  le  s create  an    n  t
  n t  e    r clause
int yearsToRetire = 0;
double newSalary = 0.0;
t    s   ar  a  le  s create  an    n  t
  n t  e    r clause

double total = 0.0;



cout << "Current age in years (1
to 64): ";
cin >> age;

if (age < 1 || age > 64)
     cout << "Please enter an age
     from 1 to 64." << endl;
else
{
     cout << "Current salary as a whole
     number: ";
     cin >> currentSalary;

   agyearsToRetire = 65 - age;
     for (double rate = 0.03;rate <
     0.06; rate += 0.01)
```

Figure 8-16 Beginning of the IPO chart information and C++ instructions for the Retirement program

Lab 8-2: Plan and Create (cont'd.)

```
                                                    {
      ass   gn current salary t    ne    salary          newSalary = currentSalary;
      ass   gn current salary t    t   tal earns  ng      total = currentSalary;

      re  eat     r (year   t   years unt  l ret  re  ent) for (int year = 2; year <=
                                                          yearsToRetire; year += 1)
                                                          {
           ne   salary   ne   salary   (1                      newSalary *= (1 + rate);
              ra   se rate)
           a      ne   salary t   t  tal earn  ngs            total += newSalary;
      en   re  eat                                        } //end for

           s  lay ra  se rate an   t  tal earn  ngs       cout << "Total with a " <<
                                                          rate * 100 <<
                                                          "% annual raise: $"
   en   re  eat                                           << total << endl;
en                                                      } //end for
                                                    }   //end if
```

Figure 8-16 Remainder of the IPO chart information and C++ instructions for the Retirement program

Lab 8-2: Plan and Create (cont'd.)

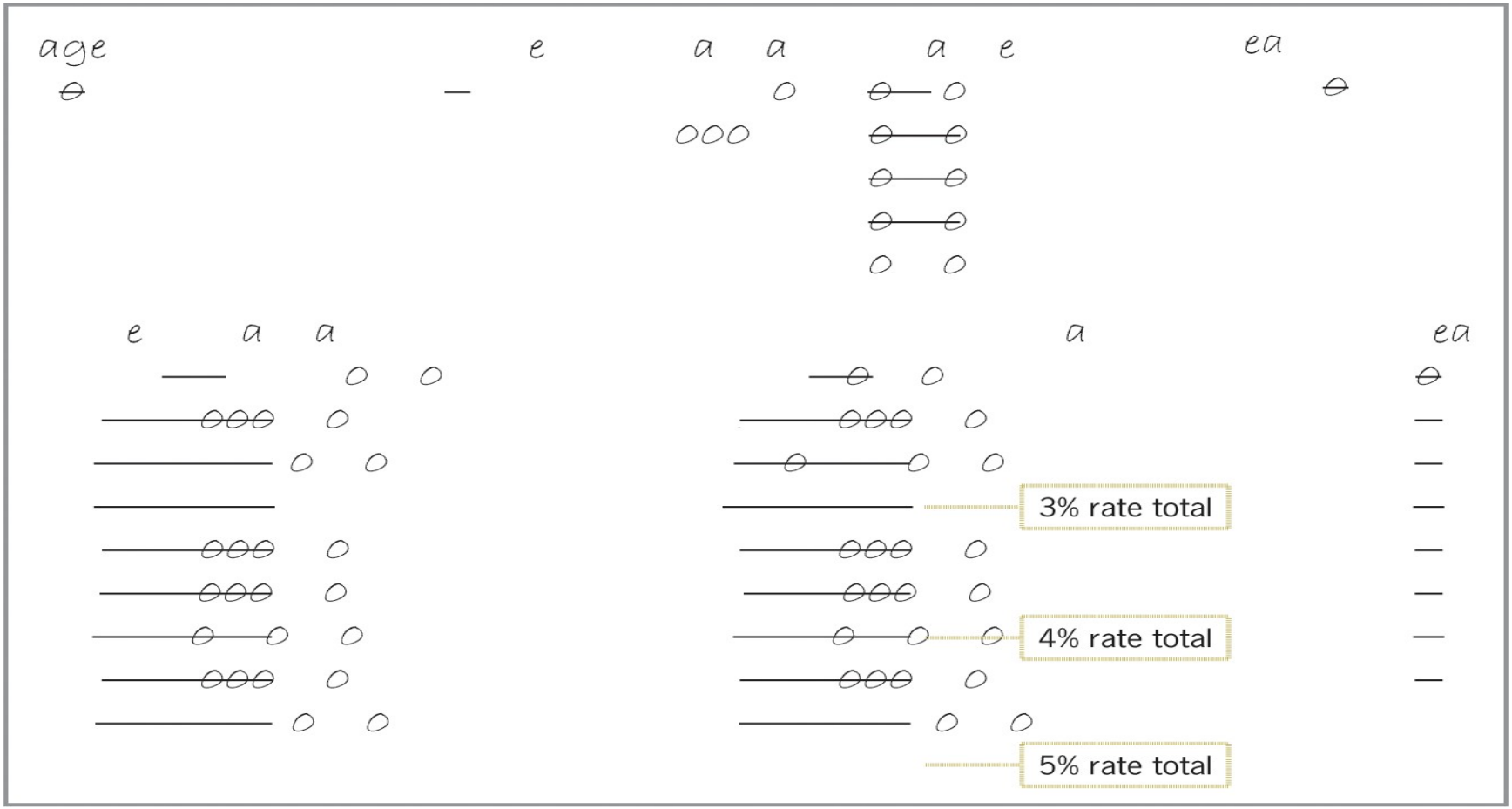

Figure 8-17 Desk-check table for the Retirement program

Lab 8-2: Plan and Create (cont'd.)

```
 1 //Lab8-2.cpp - displays a person's total
 2 //earnings before retirement at age 65,
 3 //using annual raise rates of 3%, 4%, and 5%
 4 //Created/revised by <your name> on <current date>
 5
 6 #include <iostream>
 7 #include <iomanip>
 8 using namespace std;
 9
10 int main()
11 {
12    int age = 0;
13    int currentSalary = 0;
14    int yearsToRetire = 0;
15    double newSalary = 0.0;
16    double total = 0.0;
17
18    cout << fixed << setprecision(0);
19
20    cout << "Current age in years (1 to 64): ";
21    cin >> age;
22
23    if (age < 1 || age > 64)
24        cout << "Please enter an age from 1 to 64." << endl;
25    else
26    {
27        cout << "Current salary as a whole number: ";
28        cin >> currentSalary;
29        cout << endl;
30
31        yearsToRetire = 65 - age;
32        for (double rate = 0.03; rate < 0.06; rate += 0.01)
33        {
34            newSalary = currentSalary;  //year 1 salary
35            total = currentSalary;      //year 1 salary
36            for (int year = 2; year <= yearsToRetire; year += 1)
37            {
38                newSalary *= (1 + rate);
39                total += newSalary;
```

Figure 8-18 Beginning of the finalized Retirement program

Lab 8-2: Plan and Create (cont'd.)

```
40              } //end for
41              cout << "Total with a " << rate * 100
42                   << "% annual raise: $" << total << endl;
43          } //end for
44      }  //end if
45      return 0;
46  }  //end of main function
```

Figure 8-18 Remainder of the finalized Retirement program

Lab 8-3: Modify

- Modify the program in Lab8-2.cpp. Change the outer *for* loop to a *posttest* loop. Save the modified program as Lab8-3.cpp
- Save, run, and test the program.

Lab 8-4: What's missing?

- The program is this lab should display the pattern of numbers shown in Figure 8-19 below.
- Follow the instructions for starting C++ and opening the Lab8-4.cpp file. Put the C++ instructions in the proper order, and then determine the one or more missing instructions.
- Test the program appropriately.

If the user enters the number 4 in response to the "How many rows?" prompt, the program should display the following pattern of numbers:

```
1
12
123
1234
```

Figure 8-19 Sample output for Lab 8-4

Lab 8-5: Desk-Check

- Desk-check the code in Figure 8-20
- What will the code display on the computer screen?

```
int sumX = 0;
int sumY = 0;

for (int x = 2; x < 7; x += 2)
{
      sumX += x;
      for (int y = 1; y < 4; y += 2)
            sumY += y;
      //end for
} //end for
cout << "sumX value: " << sumX << endl;
cout << "sumY value: " << sumY << endl;
```

Figure 8-20 Code for Lab 8-5

Lab 8-6: Debug

- Follow the instructions for starting C++ and opening the Lab8-6.cpp file.
- The program should display a store's quarterly sales, but it is not working properly.
- Debug the program.